2014 13th International Symposium on Parallel and Distributed Computing

Understanding and Optimizing GPU Cache Memory Performance for Compute Workloads

Kyoshin Choo, William Panlener, and Byunghyun Jang
Computer and Information Science
The University of Mississippi
University, MS 38677
{kchoo,wpanlener}@go.olemiss.edu
bjang@cs.olemiss.edu

***Abstract*—Processing elements such as CPUs and GPUs depend on cache technology to bridge the classic processor memory subsystem performance gap. As GPUs evolve into general-purpose co-processors with CPUs sharing the load, good cache design and use becomes increasingly important. While both CPUs and GPUs must cooperate and perform well, their memory access patterns are very different. On CPUs only a few threads access memory simultaneously. On GPUs, there is significantly higher memory access contention among thousands of threads. Despite such different behavior, there is little research that investigates the behavior and performance of GPU caches in depth. In this paper, we present our extensive study on the characterization and improvement of GPU cache behavior and performance for general-purpose workloads using a cycle-accurate ISA level GPU architectural simulator that models one of the latest GPU architectures, Graphics Core Next (GCN) from AMD.**

Our study makes the following observations and improvements. First, we observe that L1 vector data cache hit rate is substantially lower when compared to CPU caches. The main culprit is compulsory misses caused by lack of data reuse among massively simultaneous threads. Second, there is significant memory access contention in shared L2 data cache, accounting for up to 19% of total access for some benchmarks. This high contention remains a main performance barrier in L2 data cache even though its hit rate is high. Third, we demonstrate that memory access coalescing plays a critical role in reducing memory traffic. Finally we found that there exists inter-workgroup locality which can affect the cache behavior and performance. Our experimental results show memory performance can be improved by 1) shared L1 vector data cache where multiple compute units share a single cache to exploit inter-workgroup locality and increase data reusability, and 2) clustered workgroup scheduling where workgroups with consecutive IDs are assigned on the same compute unit.

I. Introduction

The success of General-Purpose computing on Graphics Processing Unit (GPGPU) technology has made high performance computing affordable and pervasive in platforms ranging from workstations to hand-held devices. Its cost and power efficiency (GFLOPS/$ and GFLOPS/watt) are unprecedented in the history of parallel computing. Data-parallel portions of applications are offloaded onto the GPU and easily accelerated by several orders of magnitude. When asynchronously combined with the task-parallel processing of multi-core CPU, this computing approach exploits both data and task-level parallelism available in many applications. This new computing paradigm is known as heterogeneous computing.

Memory hierarchy is an important distinguishing hardware architectural feature between a many-threaded GPU and a traditional multi-threaded CPU. Designed to process a massive number of graphics shaders usually with little control flow, GPUs shun high-capacity, sophisticated caches in favor of a much larger number of arithmetic functional units. Rather, GPUs embrace a wider memory bus with a higher bandwidth connection to off-chip memory. The resulting higher latency penalty is minimized by hiding memory latency through thread switching. Introducing multiple levels of cache between functional units and off-chip memory improves traditional memory performance, especially effective latency. Although GPUs are bandwidth optimized, low latency is highly desired in many workloads (especially non-graphics). GPU hardware architects typically include more caches to handle irregular and complicated memory access patterns.

Most modern GPUs have two-levels of cache and software-managed scratchpad memories [2], [5], [7], [9]. For example, AMD introduced a 16 KB L1 Vector Data (L1VD) cache per Compute Unit (CU) and a 384 KB to 768 KB shared L2 Data (L2D) cache in their latest GPU architecture, Graphics Core Next (GCN, a.k.a. Southern Island). NVIDIA also introduced a relatively large (up to 48 KB per stream multiprocessor) and a configurable L1 cache and shared L2 cache in their latest Kepler architecture. Although there has been research that reports the benefits of cache, there is little research that studies the cache behavior and performance from a hardware design perspective. Since the memory access patterns of compute workloads are quite different from those of traditional graphics shaders, it is unclear how caches behave and what impact each configuration would have.

In this paper, we thoroughly study the cache behavior and performance on modern GPUs using a cycle-accurate ISA-level detailed architectural simulator. The target architecture we investigate is Graphics Core Next (GCN) from AMD. Using representative compute benchmark workloads with different characteristics, we present the following findings and improvements in this paper.

- The cache hit rate of private L1VD cache is low when compared to conventional processors. Such low cache hit

978-0-7695-5265-1/14 $31.00 © 2014 IEEE
DOI 10.1109/ISPDC.2014.29

IEEE computer society

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY BOMBAY. Downloaded on January 16,2026 at 12:37:26 UTC from IEEE Xplore. Restrictions apply.

rates increase the traffic to lower level cache, degrading overall memory performance significantly.
- The majority of L1VD cache misses are compulsory misses, caused by lack of data reuse among massive active threads on a CU.
- Memory access contention[1] is significantly higher in shared L2D cache, accounting for up to 19% of total accesses in some benchmarks. Despite the high hit rate of L2D cache, high contention remains a main performance barrier at the L2 level.
- Due to massive and simultaneous thread execution and resulting memory requests, memory request coalescing plays a critical role in overall memory performance. Data requests to the same address or contiguous linear address belonging to the same cache line are fully coalesced and we demonstrate this reduces memory traffic significantly.
- We observed there is inter-workgroup locality affecting cache behavior and performance. We demonstrate that default workgroup scheduling policy (i.e., first availability basis policy) fails to exploit this workload locality. We also demonstrate shared L1VD cache helps exploit this locality and more than compensates for the additional sharing overhead. The shared coalescing unit and clustered workgroup scheduling are the primary mechanisms for exploiting this locality.

Based on the observations listed above, we demonstrate that GPU memory performance can be improved by the following modifications.

- Shared L1VD cache among multiple CUs instead of private cache reduces compulsory misses and memory contentions. It also enhances inter- as well as intra-workgroup locality, which leads to better memory access coalescing. Increasing cache line size during merging further improves coalescing.
- Clustered workgroup scheduling policy enhances inter-workgroup data locality and improves memory access coalescing among workgroups.

The remainder of the paper is structured as follows: Section II provides background on GPU hardware, OpenCL thread mapping, and the experimental framework used. Section III presents our detailed evaluation of GPU cache behavior and performance. Section IV describes two cache performance improvement techniques: shared L1 vector data cache and clustered workgroup scheduling, along with experimental results. Section V discusses related works, and we conclude the paper in Section VI.

II. Background and Experimental Framework

This section presents background on GPU architecture, thread mapping, and experimental framework used in this work.

[1]In this work, contention refers to a situation where multiple memory requests are made and only some of them are served while others are blocked and keep retrying.

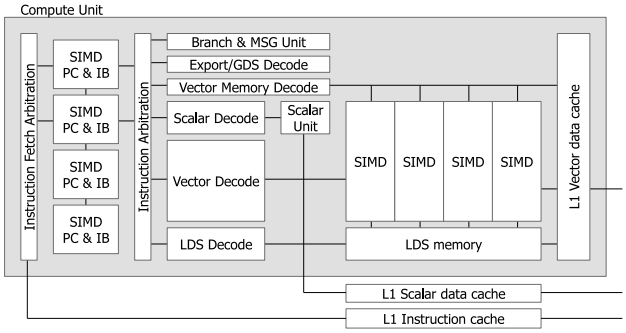


Fig. 1: GCN compute unit [2]

A. Target GPU architecture

Our target hardware under study is the GCN AMD Radeon HD7970, one of the most advanced and powerful commercial GPUs available. In order to deliver balanced performance of graphics and general-purpose compute workloads, it employs a hybrid execution engine where vector and scalar operations are executed in different hardware units [2]. The CU, a basic computational building block of GPUs, is illustrated in Fig. 1. It consists of four Single Instruction Multiple Data (SIMD) vector units, a scalar unit, a branch unit, and memory subsystems (local memory, L1 and L2 cache etc.). Each SIMD unit simultaneously executes one instruction across 16 threads (work items) per cycle, completing it in four cycles giving a wavefront of 64 threads. The scalar unit consists of a scalar ALU and an 8KB register file for executing independent operations, which include simple arithmetic and control flow operations. This prevents threads within a wavefront from executing exactly the same computation redundantly. A CU supports up to 40 wavefronts in flight and can issue up to five different types of instructions at each cycle.

The memory hierarchy of GCN is designed to better support parallelism. Each CU has a 16KB, 4-way set associative L1VD cache. Four CUs share a 16KB L1 Scalar Data (L1SD) cache and a 4-way set associative read-only 32KB L1 instruction cache. L1VD and L1SD caches are backed by a unified L2 cache of 768KB size. This L2D cache uses 16-way set associative configuration with a 64B cache line employing a Least Recently Used (LRU) replacement policy. It is physically partitioned into slices which couple them to each memory channel. A crossbar fabric from the CUs to the cache and memory partitions provides access to these slices of the L2 cache. This cache level connects to the off-chip main memory.

B. Thread configuration and mapping

In OpenCL programming, the threads are sub-grouped into workgroups (WGs) of equal size. At run time, WGs are then assigned to each CU by hardware. Once assigned, a WG runs only on the assigned CU until completion. Threads in a WG can share data through local memory or L1 cache. Once allocated, a WG is subdivided into smaller groups

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY BOMBAY. Downloaded on January 16,2026 at 12:37:26 UTC from IEEE Xplore. Restrictions apply.

TABLE I: Baseline GPU architectural configuration

Compute Unit		
# of Compute Unit on GPU		32
# of SIMD engines per CU		4
Maximum active wavefronts per CU		40
Memory Hierarchy		
L1 vector data cache (one per CU) 32 units	# of sets	64
	Associativity	4
	Line size	64B
	Latency (cycles)	1
	Total L1 cache size	512KB
L1 scalar data cache (one per 4 CUs) 8 units	# of sets	64
	Associativity	4
	Line size	64B
	Latency (cycles)	1
	Total L1 cache size	128KB
L2 unified data cache	# of sets	128
	Associativity	16
	Line size	64B
	Latency (cycles)	10
	Total L2 cache size	768KB
Global Memory	Line size	64B
	Bus width	256 bit
	Latency (cycles)	100
	Total memory size	4GB

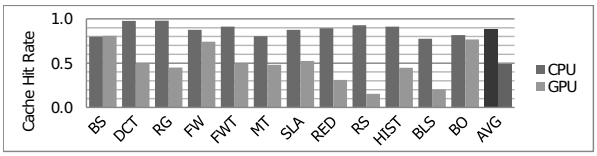
called wavefronts[2] whose size is a hardware design parameter. Because multiple WGs can be assigned to the same CU depending on hardware resource availability, multiple wavefronts from different workgroups can run on a CU at the same time. The hardware wavefront scheduler picks a ready-to-execute wavefront from the pool. This process is called thread switching.

C. Simulation and benchmarks

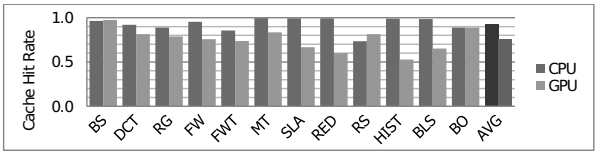
For accurate simulation, we use a fully-verified, cycle-accurate, detailed CPU-GPU heterogeneous processor architectural simulation infrastructure, called Multi2Sim [8], [12] for all experiments. Multi2Sim is an open source, modular, and fully configurable simulator that models several commercial CPUs and GPUs. Multi2Sim implements all important hardware blocks and provides detailed performance statistics for each block and memory subcomponent. The user configures Multi2Sim using a standard *ini* file format. Table I lists the baseline hardware configuration used in this study, which matches AMD HD7970 GPU specification.

We use representative benchmark workloads provided in AMD APP SDK [1]. For accurate assessment, input sizes and parameters are carefully chosen to fully occupy all CUs.

[2]It is known as warps on NVIDIA platform.



(a) L1 data cache hit rates



(b) L2 data cache hit rates

Fig. 2: Comparison of L1 and L2 data cache hit rate

III. Dissection of GPU Cache Behavior and Performance

In this section, we present our evaluation and analysis of cache behavior and performance. For comparison purposes, we simulated the serial version of benchmarks using the generic x86 model implemented in Multi2Sim. Detailed CPU configuration can be found in [8].

A. L1 vector data cache hit rate is considerably low

Fig. 2 compares L1 (vector) data cache hit rates between the CPU and GPU. The results show that, in all benchmarks, L1VD cache hit rates on the GPUs are considerably lower than those on the CPUs, 0.49 vs. 0.88 on average. L2D cache exhibits higher hit rate for both CPUs and GPUs than L1 data cache. On average, L2D cache hit rate is 0.73 on the GPUs while it is 0.93 on the CPUs. For both L1 and L2, average hit rates of the GPUs are relatively and considerably lower than those of the CPUs (by 44% and 22% respectively). Such low cache hit rates result in higher memory traffic at the lower levels of the memory hierarchy (L2 or off-chip global memory) and leads to longer memory latency, degrading the overall memory performance.

B. L1 vector and scalar data caches exhibit different cache behavior

Separate L1SD cache is used for the data requested by scalar memory instruction. Comparing with L1VD cache, its hit rate is considerably higher (96% on average as shown in Fig. 3. One outlier is *RS* benchmark. In that benchmark, the memory access stride for scalar memory instruction changes from 1 to 2^k (length of input string) with steps of multiples of 2. When memory access stride aligns with the number of sets of the cache, conflict misses occur and it lowers the cache hit rate.

We observe that L1SD cache exhibits higher hit rates because for two reasons. First, four CUs share a single cache. Such a configuration significantly increases inter-thread locality. Second, scalar operations have both temporal and spatial locality because scalar memory instructions are typically used

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY BOMBAY. Downloaded on January 16,2026 at 12:37:26 UTC from IEEE Xplore. Restrictions apply.

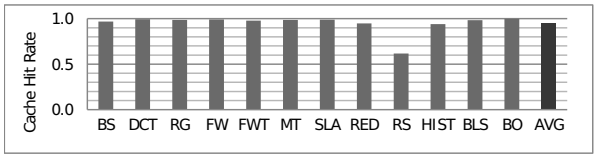


Fig. 3: Cache hit rates for GPU L1SD cache

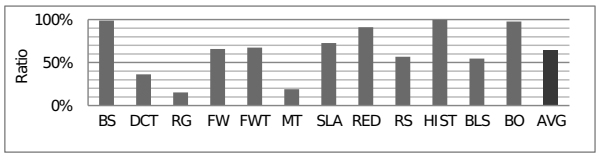


Fig. 4: Contribution of compulsory miss in L1VD cache miss

for accessing common memory buffer description or uniform[3] data.

C. Compulsory miss dominates L1 vector data cache misses

According to our cache miss classification, compulsory misses are a major contributor to the high L1VD cache miss as Fig. 4 shows. On average, compulsory misses account for 65% of total misses. This behavior is contradictory to the conventional wisdom that compulsory misses are negligibly small on traditional multi-core processors [10].

Although the OpenCL threading model expects independent data, there is a certain degree of data reuse between the threads across different CUs. For example, consider the DCT benchmark which uses matrix multiplication. Fig. 5 shows the data use patterns of three arrays. Assuming that there are 16 CUs ($CU0 \sim CU15$) available and 16 WGs total ($WG0 \sim WG15$) to execute, one WG is assigned to each CU. Suppose that $WG0$ calculating C_{00} is assigned to $CU0$, $WG1$ calculating C_{01} is assigned to $CU1$, and so on. $CU0$ uses data A_{00}, A_{01}, A_{02}, A_{03}, B_{00}, B_{10}, B_{20}, and B_{30}, and CU1 uses the data A_{00}, A_{01}, A_{02}, A_{03}, B_{01}, B_{11}, B_{21}, and B_{31}. In this scenario, $CU0$, $CU1$, $CU2$ and $CU3$ use the same data A_{00}, A_{01}, A_{02}, A_{03}. When such data access patterns occur across many CUs, requests for every first access of the same data by each CU result compulsory misses on each CU. For the RG and MT benchmarks, contribution of compulsory miss is relatively smaller than other benchmarks as data reuse of pre-loaded data is relatively higher.

D. L2 data cache access contention is significant

Our detailed profiling shows that there is significant contention in L2D for two main reasons. First, shared L2D caches receive a high volume of simultaneous memory requests from multiple CUs caused by high L1VD misses. This is inevitable unless the L2D level supports as many ports as the number of CUs. Implementing many-ports, however, not only is costly but also fails to handle cache misses and memory writes. Second, the latency of off-chip global memory access is long.

[3]Uniform access refers to the situation where all threads within a wavefront behave in the same way (e.g., accessing the same data address).

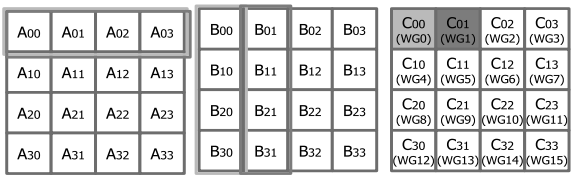


Fig. 5: Data usage in matrix calculation of DCT

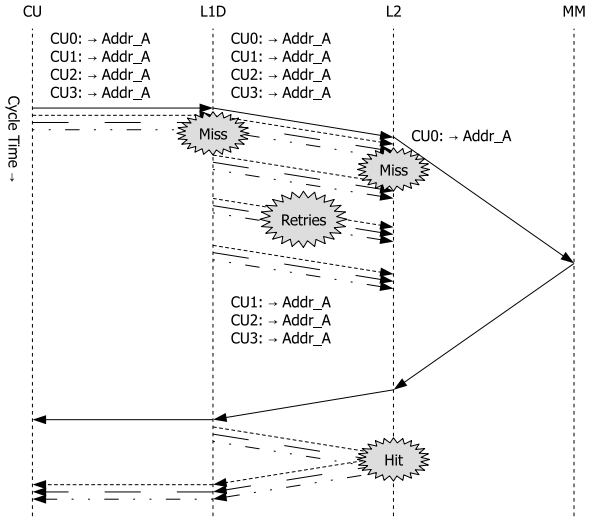


Fig. 6: Illustration of L2D cache memory access contention on GPUs.

In the case of misses, all other threads requesting L2D cache lines except the active thread have to retry while the requested cache line is locked by the active thread.

Fig. 6 illustrates memory access contention. First, wavefronts from different CUs request data with the same memory addresses in this example to its own private cache. If they all miss, then they request to the lower level L2D cache simultaneously or within a very small time interval. Given these multiple requests, one of them ($CU0$ in this example) succeeds and locks L2D cache. The rest of the requests from $CU1$, $CU2$, and $CU3$ are blocked and continue to retry until L2D cache is released. Due to the long latency of main memory, many retries occur. Once $CU0$ releases L2, the retries from $CU1$, $CU2$, and $CU3$ succeed with hits and the contention ends. Fig. 7 shows the contribution of contention traffic to the total L2 cache traffic in the DCT benchmark. 19% of total L2 cache traffic is caused by contention in the DCT benchmark and average contention traffic over all benchmarks is 8%.

E. Coalescing occurs massively

GPU achieves high performance through massively parallel thread execution. Such massive thread execution subsequently issues data requests to its own L1 cache unit. GPU memory subsystems are equipped with a hardware block called a *coalesce unit* in L1VD cache as shown in Fig. 8. The

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY BOMBAY. Downloaded on January 16,2026 at 12:37:26 UTC from IEEE Xplore. Restrictions apply.

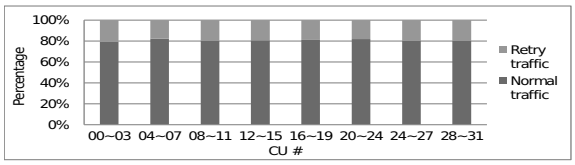


Fig. 7: Contention traffic in L2 cache in DCT benchmark

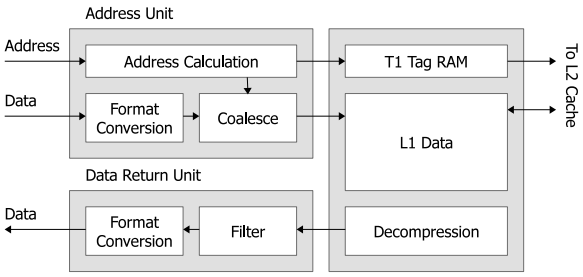
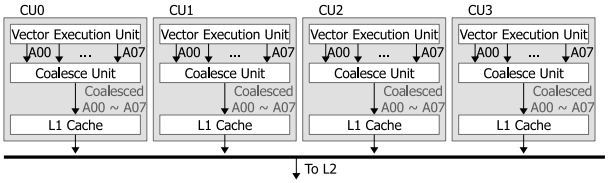


Fig. 8: AMD Southern Island L1VD cache unit structure [2]

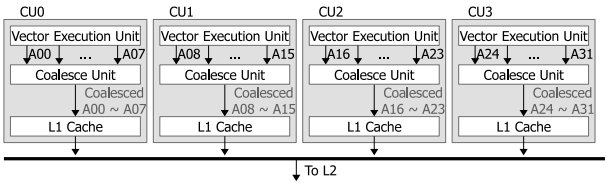
coalesce unit merges data requests for the same cache line until the lower level cache responds. Our experiment shows that such coalescing occurs massively and plays a critical role in reducing memory traffic.

Fig. 9 illustrates two coalescing cases. Fig. 9a shows a case where each CU issues memory requests for the same address. In this case, memory requests of address $A00 \sim A07$ are merged together into a single request. Cache line size is a major factor for determining the degree of coalescing. If the range of requested addresses exceeds the cache line size, requests are divided into several partially-coalesced requests. Fig. 9b shows another case where each CU issues memory requests for contiguous and linear addresses ($A00 \sim A07$ from $CU0$, $A08 \sim A15$ from $CU1$ and so on). Requests $A00 \sim A07$ are coalesced to a single request for $CU0$, and requests $A08 \sim A15$ are coalesced to a single request for $CU1$, and so on. These coalesced requests can be further coalesced together if the cache line size of L2D cache is larger than that of L1VD cache.

Fig. 10 shows the coalescing degree for each benchmark. The coalescing degree is defined as the ratio of the number of coalesced requests to the total L1D requests. As can be seen, on some benchmarks, the coalescing degree is very high, more than 30 of requests are coalesced together. On average, about 14 of data requests belong to the same cache line and those requests are coalesced together to make one request to the lower level cache. High coalescing degree benefits from larger cache line size since for larger cache line, there is more chance for the requests to be coalesced together, therefore, reduces the memory traffic. Since memory access patterns in current GPGPU applications are usually regular, larger cache line size with high coalescing degree uses cached data effectively without waste.



(a) Same address



(b) Contiguous and linear address

Fig. 9: Two example cases of memory coalescing

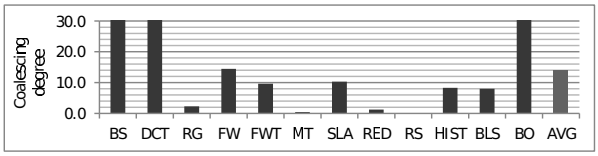


Fig. 10: Average coalescing degree at L1VD cache

F. Workgroup assignment impacts locality

Our study shows that there exists *intra-workgroup* and *inter-workgroup* localities. The former can be exploited by a programmer via thread indexing while the latter by the hardware's workgroup assignment policy. To the best of our knowledge, the default workgroup assignment policy employed by all modern GPUs is to assign workgroups to the first available CU (so called *first availability basis*). This guarantees good CU utilization but may miss an opportunity to exploit inter-workgroup locality. Our experiment demonstrates that an alternate scheme whereby contiguous workgroups are assigned to the same CU can improve L1VD cache hit rate by up to 17%. Fig. 5 shows the access patterns resulting in higher inter-workgroup locality. When $WG0 \sim WG3$ were assigned on the same CU, WGs, $WG0 \sim WG3$, would reuse the first row ($A00 \sim A03$) of array A, and WGs, $WG4 \sim WG7$ would reuse the second row ($A10 \sim A13$). Note that array B would not make any significant difference between the two cases.

IV. Improving Cache Performance

Based on our observation and understanding of cache behavior and performance discussed in the previous section, we demonstrate that two approaches, *shared L1 vector data cache* and *clustered workgroup scheduling*, improve GPU memory system performance.

A. Shared L1 vector data cache

We have shown that the majority of the cache misses in L1VD cache are compulsory misses. A compulsory miss is

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY BOMBAY. Downloaded on January 16,2026 at 12:37:26 UTC from IEEE Xplore. Restrictions apply.

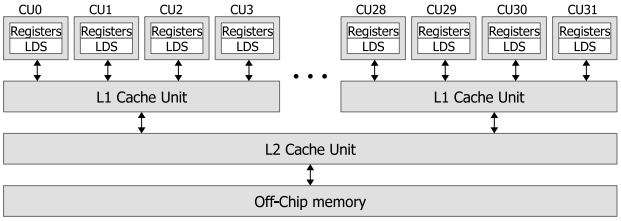


Fig. 11: Example of shared L1VD cache with the sharing factor of 4

usually hard to avoid in traditional task-parallel CPU architectures because data from/to memory is both independent and exclusive to each task (i.e., little data sharing or locality between tasks). GPUs, on the other hand, possess a great deal of data reuse among threads (e.g., wavefronts and workgroups) - the same data sections are used by multiple threads, intra-thread or inter-thread. We propose a L1VD cache shared by multiple CUs to exploit this intra/inter-thread locality.

Fig. 11 illustrates our proposed L1VD cache structure with 4 CUs sharing a single L1VD cache (i.e., sharing factor of 4). While data requests from active wavefronts in a single CU are coalesced together in private L1VD cache, shared L1VD cache allows data requests from multiple CUs to be coalesced together. To fully benefit, the coalesce unit in each shared L1VD cache needs multiple read/write ports to interface with the CUs. Since this only increases requests in coalesce unit with reasonable number of coalescing degree, the overall complexity increase is less than linear by the sharing factor. Memory requests from multiple CUs are better coalesced together at coalesce unit, and the coalesced requests will be sent to the L1 cache memory unit.

As requests from multiple CUs are merged, access contention (and thus total number of memory accesses) at lower level of memory hierarchy substantially decreases as well. Cache hit rates increase with higher sharing factors but relative benefit is limited by increasing implementation costs.

For example, when merging L1VD caches with a sharing factor of 4, the merged cache size of shared L1VD cache is 4 times the original size. This merging can be implemented by increasing either the number of sets, associativity or cache line size. Increasing the number of sets or associativity increases cache latency for both hit and misses due to associated overhead, while increasing cache line up to the memory bus width size does not. As discussed in Section III-E, larger cache line size also helps increase coalescing degree, thus reducing memory traffic.

Fig. 12 shows the experimental results for various benchmarks with five different sharing factors. As shown in Fig. 12a, hit rates increase as the sharing factor increases. In most benchmarks (11 out of 12), the cache hit rate has been improved over all sharing factors. However, in one benchmark, HIST, the hit rate decreases as the sharing factor increases. For that case, despite the lower hit rate, total traffic to the lower level caches has been decreased and the overall execution cycle has improved. Fig. 12b represents total execution cycles improvement. To increase cache line size for L2D cache with larger sharing factor, the number of sets or associativity is reduced, being smaller than the baseline. Conflict miss starts to increase with sharing factor of 8 or larger on benchmarks BS and MT. For benchmark RS, low coalescing factor worsens the performance of total execution time.

TABLE II: Performance improvements by various sharing factors.

Improvements on L1VD cache hit rates				
	2	4	8	16
BS	9%	15%	19%	20%
DCT	16%	19%	20%	26%
RG	15%	29%	48%	56%
FW	5%	7%	9%	9%
FWT	16%	24%	30%	31%
MT	17%	40%	66%	72%
SLA	4%	10%	10%	12%
RED	10%	17%	24%	21%
RS	39%	197%	319%	372%
HIST	-4%	-8%	-20%	-29%
BLS	31%	60%	74%	107%
BO	6%	9%	11%	14%
Improvements on total execution time				
	2	4	8	16
BS	11%	11%	-17%	-86%
DCT	10%	17%	22%	30%
RG	-2%	-3%	-6%	-8%
FW	33%	41%	40%	30%
FWT	42%	60%	59%	61%
MT	9%	11%	-23%	-55%
SLA	5%	5%	6%	6%
RED	16%	26%	25%	28%
RS	-12%	-31%	-45%	-237%
HIST	0%	0%	3%	5%
BLS	2%	3%	1%	2%
BO	0%	0%	0%	0%

Fig. 13 shows the performance improvement with sharing factor 4. 11 out of 12 benchmarks have been improved. Only one benchmark, RS, has negative impact as its memory access pattern results a lower coalescing degree. Overall, a sharing factor of 4 improves hit rate by 20% and the total execution cycle time by 12% on average. Table II shows the detailed statistics for all sharing factors.

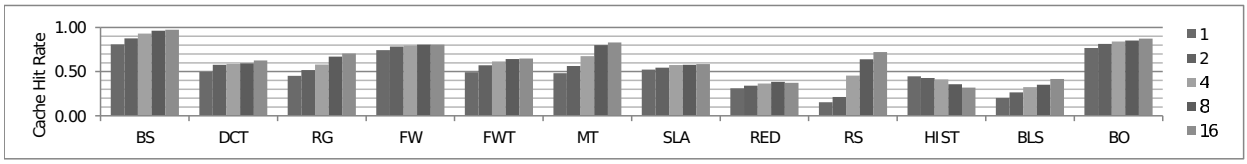
B. Clustered workgroup scheduling

Motivated by inter-workgroup locality, we propose a workgroup scheduling (assignment) policy where workgroups with contiguous workgroup IDs are assigned to the same CU. Given a workgroup ID, the destination CU is determined by:

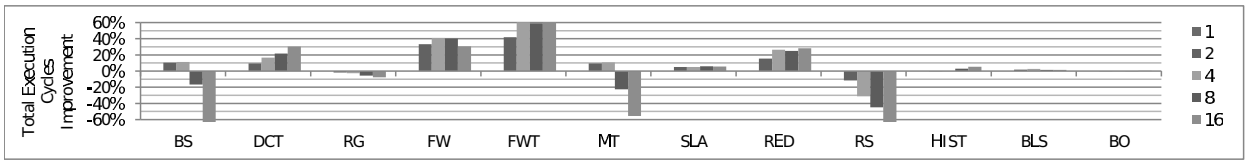
$$CU_{ID} = \frac{WG_{ID}}{\left\lfloor \frac{|WG|+|C|-1}{|C|} \right\rfloor}$$

where CU_{ID} is a CU ID, WG_{ID} is a workgroup ID, $|WG|$ is the number of workgroups, and $|C|$ is the number of CUs.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY BOMBAY. Downloaded on January 16,2026 at 12:37:26 UTC from IEEE Xplore. Restrictions apply.



(a) L1VD cache hit rates



(b) Total execution cycles improvement

Fig. 12: Performance results for various sharing factors

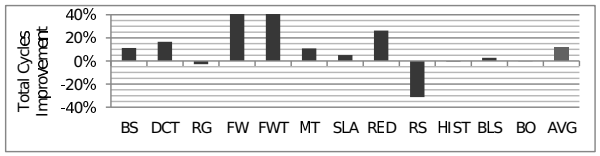


Fig. 13: Total execution cycles improvement for sharing factor of 4

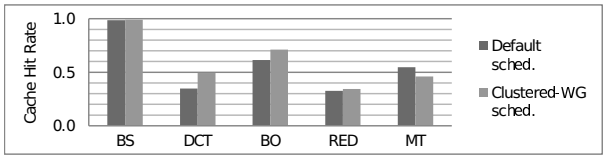


Fig. 14: L1VD cache hit rate for clustered workgroup scheduling

To avoid any undesired idle CUs, the idea of "workgroup borrowing" is implemented as shown in Algorithm 1. An idle CU will take half of the inactive workgroups from the queue of non-idling CU. This will lead to some thrashing of the workgroup queue as very few workgroups remain inactive near the end of kernel execution, but this is a minimal boundary effect assuming large input sizes.

Algorithm 1 Choosing next workgroup to activate on a CU

```
for all compute units C do
  if C is available then
    if C's queue is empty then
      Borrow half of the workgroups from the next non-idle CU
    end if
    if C's queue is non-empty then
      Activate and remove a workgroup from C's queue
    end if
  end if
end for
```

To investigate the impact of workgroup clustering with borrowing policy, five benchmarks with enough workgroups to simulate this scheme were carefully chosen from the AMD APP SDK. The results of the experiments are illustrated in Fig. 14. This technique improves or at least does not degrade the cache hit rate for 4 of 5 benchmarks. One benchmark is impacted negatively. The analysis of that individual kernel reveals that positive impact is associated with kernels that have a relatively small access stride (therefore more inter-workgroup locality), and negative impact is associated with kernels that have a relatively large access stride (less inter-workgroup locality).

V. Related Works

Despite a plethora of GPU memory optimization studies on high-level programming [3], [6], [11], [13], [14], there is little research dealing with the behavior or performance of the GPU memory hierarchy from a hardware architecture perspective. We believe this is due to the absence of effective tools or methods such as a detailed architectural simulator or hardware profiler allowing pinpoint collection of useful performance data.

Choi [4] proposed two cache management schemes to reduce off-chip memory access. One scheme is write-buffering that tries to utilize the shared cache for inter-block communication instead of off-chip DRAM access, and the other is read-bypassing which prevents the shared cache from being polluted by streamed data that are consumed only within a thread-block. Their work, however, is different from ours in two aspects. First, they used a simulator at intermediate language level (NVIDIA's PTX), and although the simulation at such level can provide useful insights, it often misses the real behavior or performance of detailed hardware structures. Second, our work focuses on more classical, fundamental cache configuration that hardware designer must know, rather than introducing additional hardware blocks on top of default cache configurations.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY BOMBAY. Downloaded on January 16,2026 at 12:37:26 UTC from IEEE Xplore. Restrictions apply.

Jia [7] investigated memory traffic on a NVIDIA GPU and characterized them into a taxonomy where three types of locality are introduced: within-warp locality, within-block locality, and cross-instruction data reuse. Based on this classification, they proposed a compile-time algorithm that determines whether or not to use L1 cache. Their approach is quite different from ours in that they develop a software algorithm to turn on and off the use of caches, whereas we understand and optimize cache behavior and performance.

To the best of our knowledge, there is no research work that details accurate cache behavior and performance on emerging GPUs. Our study characterizes cache level memory traffic from a massively parallel thread execution standpoint, and proposes a better cache configuration as well as workgroup scheduling policy to improve the memory performance of modern GPUs.

VI. Conclusions

As cache plays more important role in GPU performance, hardware designers should understand the exact behavior and performance of cache systems. Programmers also need to know the impact of different memory access patterns on memory performance to better optimize their kernels.

In this paper, we characterize and analyze cache level memory traffic using a cycle accurate architectural simulator. Our study reports several interesting observations and improvements. Cache hit rates are noticeably lower on GPUs than on CPUs. Private L1VD caches especially suffer from higher miss rates. A major culprit is compulsory misses due to low data reuse. Because of the private nature of L1VD cache, inter-workgroup data reusability rate gets lower, and thereby causes many compulsory misses. This private nature also causes higher contention in L2D cache and unnecessary memory request traffic to lower level of the memory hierarchy.

Parallel Single Instruction Multiple Thread (SIMT) execution model enables massive memory coalescing. Due to inter-workgroup and intra-workgroup locality, many threads request the same or nearby data to lower level memory hierarchy and these access patterns do not benefit from private caching. Current round-robin workgroup scheduling also fails to exploit this type of locality.

To cope with the limited locality and reusability of data, we present multi-CU shared L1VD cache scheme and clustered workgroup scheduling. Our experiment demonstrates that these schemes increase inter-workgroup locality and, as a result, reduce compulsory misses, contentions in L2D cache. This more than compensates for the additional overhead of the shared L1VD cache.

Acknowledgment

The authors would like to thank Dana Schaa and Rafael Ubal for their valuable help on Multi2Sim. This research is supported in part by National Science Foundation (NSF) grant CCF-1337138 and MediaTek Inc.

References

[1] AMD, Inc. AMD Accelerated Parallel Processing (APP) Software Deve lopment Kit (SDK). http://developer.amd.com/sdks/amdappsdk/.

[2] AMD, Inc. AMD Graphics Cores Next (GCN) Architecture, June 2012. http://www.amd.com/jp/Documents/GCN_Architecture_whitepaper.pdf.

[3] M. M. Baskaran, U. Bondhugula, S. Krishnamoorthy, J. Ramanujam, A. Rountev, and P. Sadayappan. A compiler framework for optimization of affine loop nests for gpgpus. In *Proceedings of the 22nd annual international conference on Supercomputing*, ICS '08, pages 225–234, New York, NY, USA, 2008. ACM.

[4] H. Choi, J. Ahn, and W. Sung. Reducing off-chip memory traffic by selective cache management scheme in gpgpus. In *Proceedings of the 5th Annual Workshop on General Purpose Processing with Graphics Processing Units*, GPGPU-5, pages 110–119, New York, NY, USA, 2012. ACM.

[5] M. Gebhart, D. R. Johnson, D. Tarjan, S. W. Keckler, W. J. Dally, E. Lindholm, and K. Skadron. Energy-efficient mechanisms for managing thread context in throughput processors. *SIGARCH Comput. Archit. News*, 39(3):235–246, June 2011.

[6] B. Jang, D. Schaa, P. Mistry, and D. Kaeli. Exploiting memory access patterns to improve memory performance in data-parallel architectures. *Parallel and Distributed Systems, IEEE Transactions on*, 22(1):105–118, Jan. 2011.

[7] W. Jia, K. A. Shaw, and M. Martonosi. Characterizing and improving the use of demand-fetched caches in gpus. In *Proceedings of the 26th ACM international conference on Supercomputing*, ICS '12, pages 15–24, New York, NY, USA, 2012. ACM.

[8] Multi2Sim Developer Team. The Multi2Sim Simulation Framework: a CPU-GPU Model for Heterogeneous Computing. http://www.multi2sim.org/.

[9] NVIDIA Corporation. NVIDIA's Next Generation CUDA Compute Architecture: Fermi, 2009. http://www.nvidia.com/content/PDF/fermi_white_papers/NVIDIA_Fermi_Compute_Architecture_Whitepaper.pdf.

[10] L. Roh and W. Najjar. Design of storage hierarchy in multithreaded architectures. In *Microarchitecture, 1995., Proceedings of the 28th Annual International Symposium on*, pages 271–278, Nov-1 Dec.

[11] I.-J. Sung, J. A. Stratton, and W.-M. W. Hwu. Data layout transformation exploiting memory-level parallelism in structured grid many-core applications. In *Proceedings of the 19th international conference on Parallel architectures and compilation techniques*, PACT '10, pages 513–522, New York, NY, USA, 2010. ACM.

[12] R. Ubal, B. Jang, P. Mistry, D. Schaa, and D. Kaeli. Multi2sim: a simulation framework for cpu-gpu computing. In *Proceedings of the 21st international conference on Parallel architectures and compilation techniques*, PACT '12, pages 335–344, New York, NY, USA, 2012. ACM.

[13] Y. Yang, P. Xiang, J. Kong, and H. Zhou. A gpgpu compiler for memory optimization and parallelism management. In *Proceedings of the 2010 ACM SIGPLAN conference on Programming language design and implementation*, PLDI '10, pages 86–97, New York, NY, USA, 2010. ACM.

[14] E. Z. Zhang, Y. Jiang, Z. Guo, and X. Shen. Streamlining gpu applications on the fly: thread divergence elimination through runtime thread-data remapping. In *Proceedings of the 24th ACM International Conference on Supercomputing*, ICS '10, pages 115–126, New York, NY, USA, 2010. ACM.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY BOMBAY. Downloaded on January 16,2026 at 12:37:26 UTC from IEEE Xplore. Restrictions apply.